

DSA ROADMAP — STUDY COMPANION

Logic First: Binary Search

The next topic, laid out the same way as before — plain-English scenarios and reasoning steps, no code — so the logic is settled before we touch a single line of Python.

Covers: Binary Search · Search a 2D Matrix · Find Minimum in Rotated Sorted Array · Search in Rotated Sorted Array · Koko Eating Bananas
5 problems — read this before our next session on syntax.

How to use this

Same drill as the last one: read the scenario, try to say out loud what you'd need to track and decide before looking at the steps, then check yourself against the steps. Nothing here is Python — it's the reasoning only. When we sit down for the syntax pass, you'll already know what the code needs to do; we'll just be learning how to say it.

Binary Search introduces one idea that's easy to miss if you only ever picture "searching a sorted array": the thing you're cutting in half doesn't have to be an array at all. Sometimes it's a RANGE OF POSSIBLE ANSWERS — you're not searching for a value's position, you're searching for the smallest or largest number that satisfies some condition. Koko Eating Bananas at the end of this set is exactly that variant, and it's worth paying close attention to how differently it's framed from the first three.

CONTENTS

1. Binary Search
2. Search a 2D Matrix
3. Find Minimum in Rotated Sorted Array
4. Search in Rotated Sorted Array
5. Koko Eating Bananas

Binary Search

KEEP IN MIND

Ask yourself: is there some ordering here that lets me throw away HALF of what's left, with certainty, every single time I check one point? That ordering is usually "the array is sorted" — but it doesn't have to be an array at all. It can be a range of possible answers with a "yes, this works / no, this doesn't" property that flips exactly once as you slide across the range. Either way, the moment you can rule out half the remaining possibilities from a single check, you've got $O(\log n)$ instead of $O(n)$ — checking one point kills half the haystack, not just one straw.

5.1 Binary Search

THE SCENARIO

You're looking up a word in a huge, already-alphabetized dictionary. You don't start at page one — you flip to roughly the middle, see whether your word comes before or after that page, and throw away the half of the book that can't possibly contain it. Repeat.

WHAT YOU'RE ACTUALLY FIGURING OUT

Given the middle value of whatever range I still have left, does my target come before it, after it, or is it the value itself?

STEPS TO TRACE THROUGH

1. Mark the very first and very last positions of the sorted list as the boundaries of what's still "in play."
 2. While there's still at least one position between (and including) those boundaries: look at the position exactly in the middle of them.
 3. **If** the middle value is exactly your target — you're done, that's the answer.
 4. **If** the middle value is smaller than your target, the target (if it exists) must be somewhere to the right — move your left boundary to just past the middle, discarding everything at or before it.
 5. **If** the middle value is bigger than your target, the target must be to the left — move your right boundary to just before the middle, discarding everything at or after it.
 6. Repeat with the new, smaller range. If your boundaries ever cross with no match found, the target isn't in the list.
-

5.2 Search a 2D Matrix

THE SCENARIO

You have a grid of numbers, arranged so that each row is sorted left to right, AND the first number of every row is bigger than the last number of the row above it. In other words, if you read the whole grid row by row, top to bottom, left to right, it's really just one long sorted list that's been folded into rows.

WHAT YOU'RE ACTUALLY FIGURING OUT

Can I treat this grid as one single sorted list without physically rebuilding it into one — by working out, for any "position number" in that imagined single list, which row and column it actually corresponds to?

STEPS TO TRACE THROUGH

1. Figure out how many total numbers are in the grid (rows times columns) — this is the length of your imagined single sorted list.
 2. Run ordinary binary search (same as 5.1) over positions 0 through that total count minus one.
 3. Whenever you need to check "what value lives at imagined position P," convert P into an actual row and column: the row is how many full rows of width fit into P, and the column is what's left over after removing those full rows.
 4. Look up the grid value at that row and column, and compare it to your target exactly like ordinary binary search — shrink left or right accordingly.
 5. Continue until found, or the range is exhausted.
-

5.3 Find Minimum in Rotated Sorted Array

THE SCENARIO

You have a sorted list that's been "cut" at some unknown point and the two pieces swapped — like taking a sorted deck, cutting it partway through, and putting the bottom half on top. You want to find the smallest number, which sits right at that cut point.

WHAT YOU'RE ACTUALLY FIGURING OUT

Looking at the middle of my current range, and comparing it to the rightmost value in that range, can I tell whether the cut point (the minimum) is in the left half or the right half?

STEPS TO TRACE THROUGH

1. Mark the first and last positions of the list as your boundaries.
 2. While your left boundary is still before your right boundary: look at the middle position.
 3. Compare the middle value to the value at your RIGHT boundary. **If the middle value is bigger** than the rightmost value, the cut point must be somewhere to the right of the middle — move your left boundary to just past the middle.
 4. **If the middle value is smaller than or equal to** the rightmost value, the right-hand side is "clean" (properly sorted with no cut in it), so the cut point must be at the middle or to its left — move your right boundary to the middle itself (not past it, since the middle could BE the answer).
 5. When your boundaries finally meet, that position holds the minimum.
-

5.4 Search in Rotated Sorted Array

THE SCENARIO

Same cut-and-swapped sorted list as before, but now you're searching for a specific target value, not just the minimum.

WHAT YOU'RE ACTUALLY FIGURING OUT

Since the whole list isn't sorted anymore, but at least ONE of the two halves around my current middle always still is — which half is the properly sorted one this time, and does my target actually fall inside that sorted half's range?

STEPS TO TRACE THROUGH

1. Mark the first and last positions as your boundaries.
 2. While your boundaries haven't crossed: look at the middle position. **If it's your target**, you're done.
 3. Work out which side is the "clean," properly sorted half: **if the value at your left boundary is less than or equal to** the middle value, the LEFT half (left boundary through middle) is the sorted one. Otherwise, the RIGHT half (middle through right boundary) is the sorted one.
 4. **If the left half is the sorted one**: check whether your target falls between the left boundary's value and the middle value. If it does, move your right boundary to just before the middle (search the left half). If it doesn't, move your left boundary to just past the middle (the target must be in the messier right half).
 5. **If the right half is the sorted one instead**: check whether your target falls between the middle value and the right boundary's value. If it does, move your left boundary to just past the middle. If it doesn't, move your right boundary to just before the middle.
 6. Repeat until found, or the boundaries cross (target isn't in the list).
-

5.5 Koko Eating Bananas

THE SCENARIO

There are several piles of bananas, and Koko needs to eat all of them within a fixed number of hours. She picks one constant "bananas per hour" eating speed for the whole time and sticks with it — if a pile has fewer bananas than her speed, she finishes it early and just waits for the next hour rather than starting the next pile. You want the SLOWEST speed she could pick and still finish in time.

WHAT YOU'RE ACTUALLY FIGURING OUT

This is different from the others — I'm not searching through the banana piles at all. I'm searching through the RANGE OF POSSIBLE EATING SPEEDS (from 1, up to the size of the biggest pile) for the smallest one that still finishes on time. Notice: if some speed S is fast enough to finish in time, then any speed faster than S is obviously also fast enough — that "once true, stays true" property is what makes this searchable in the first place.

STEPS TO TRACE THROUGH

1. Set your range of possible speeds from 1 (slowest conceivable) up to the size of the single largest pile (eating any faster than that on any one pile is wasted speed).
2. While your low and high boundaries haven't crossed: try the speed exactly in the middle of that range.
3. Work out how many total hours it would take to finish every pile at that middle speed — for each pile, that's the pile size divided by the speed, rounded UP (since a partial hour still costs a full hour).
4. **If** that total is within the allowed number of hours, this speed **WORKS** — but it might not be the slowest that works, so remember it as your best answer so far, and try searching the **SLOWER** half (move your high boundary down to just below the middle).
5. **If** that total goes over the allowed hours, this speed is too slow — you need to search the **FASTER** half instead (move your low boundary up to just above the middle).
6. Once your boundaries cross, the best working speed you recorded is the answer.

STILL TO DO (NEXT ROUND)

Time Based Key-Value Store — a design problem (same flavor as Min Stack) that uses binary search over a sorted list of timestamped entries. Median of Two Sorted Arrays — the hardest problem in this topic, worth leaving until the rest of this set feels solid.